

```

/*:
## App Exercise – Similar Workouts

>These exercises reinforce Swift concepts in the context of a fitness
tracking app.

Previously in app exercises, you've used inheritance to enforce similar
behavior between different kinds of workouts. However, you can enforce this
behavior between different workouts using protocols.

Create a protocol `Workout` with two requirements: a settable property called
`distance` and a settable property called `time`, both of type `Double`.
*/
protocol Workout {
    var distance: Double { get set }
    var time: Double {get set}
}

//: Create two structs, `RunningWorkout` and `SwimmingWorkout`. Both should
conform to the `Workout` protocol. Uncomment the function
`simpleWorkoutDescription` below, create an instance of each of your structs,
and call `simpleWorkoutDescription` twice, passing in a `RunningWorkout`
object and then a `SwimmingWorkout` object.
func simpleWorkoutDescription(workout: Workout) {
    print("You went \((workout.distance) meters in \((workout.time) seconds.")
}

struct RunningWorkout: Workout {
    var distance: Double
    var time: Double
}

let r1 = RunningWorkout(distance: 500.2, time: 30.5)

struct SwimmingWorkout: Workout {
    var distance: Double
    var time: Double
}

let s1 = SwimmingWorkout(distance: 300.2, time: 20.5)

simpleWorkoutDescription(workout: s1)
simpleWorkoutDescription(workout: r1)

/*:
[Previous](@previous) | page 4 of 5 | [Next: App Exercise – Heart Rate
Delegate](@next)
*/

```